CISC 3130 Homework: Shah Rayan

Part A: Time Complexity

Method	Structures it touches	Big-O	One-line reason
addSong	Library, playlist, undo stack	O(1)	Put the id in the map and link a new node at the tail, no walking needed.
findSong	Library	O(1)	One lookup in the hash map.
play	Library, history, play counts, recent deque	O(1)	A few map lookups and one push. The recent deque only holds 10 songs, so scanning it is constant.
queueLater	Library, up next	O(1)	Look up the song, then addLast.
queueNext	Library, up next	O(1)	Look up the song, then addFirst.
next	Up next, library, playlist, history, play counts, recent deque	O(k + 1)	Might pop k removed songs off up next first, then follows node.next.
back	History, library, up next, play counts, recent deque	O(k + 1)	Might pop k removed songs off history before it finds a live one.
removeSong	Library, playlist, play counts, recent deque, undo stack	O(1)	Get the node from the map, unlink it, delete the map entries. The recent scan is capped at 10.
showPlaylist	Playlist	O(n)	Has to walk every node to collect the titles.

PART B: 1. Playlist (my own doubly linked list)

The playlist keeps the songs in order so next() can move forward. I made the library store each song's node, so when I remove a song I can grab its node and fix its .prev and .next links right away, which is O(1). With a plain array I'd have to shift everything after the removed song, and that's O(n). Java's LinkedList wouldn't work as well either, because it doesn't give you its nodes, so removing by song name means searching for the song first. The downside of building my own was handling the four removal cases (only 1 node, head, tail, middle), but that's what makes removal fast.

2. Library and play counts (my own hash map)

The library lets me find any song by id without scanning the playlist, and a second map keeps the play counts. That's what keeps play, queueNext, queueLater, findSong and removeSong fast. If I used an array, every one of those would be an O(n) search. Java's HashMap would be about as fast with a lot less code. I built my own so I'd have to deal with hashing, collisions, removing from a chain, and resizing myself. The cost is more code and my map only takes String as keys.